

PROCESAMIENTO DE DATOS MASIVOS

Actividad 2 – Programación en Spark

Ejercicio 3

Enunciado

(Dado un dataset que contenga entradas con la forma “persona;método_pago;dinero_gastado”, crea un programa llamado `personaYMetodosDePago` que:

- Por cada persona indique en cuántas compras pagó más de 1500 euros con tarjeta de crédito. La solución se tiene que guardar en una carpeta llamada `comprasConTDCMayorDe1500`.
- Por cada persona indique en cuántas compras pagó menos o igual a 1500 euros con tarjeta de crédito. La solución se tiene que guardar en una carpeta llamada `comprasConTDCMenorIgualDe1500`.

Solución

Se propone la solución contenida en la carpeta “ej3”, compuesta por los siguientes documentos:

- Datos de entrada: `entrada.txt`
- Código: se han desarrollado el script `personaYMetodosDePago.py`
- Comando de ejecución: el código a escribir en la terminal se reporta en `terminal_personaYMetodosDePago.txt`
- Resultado: el resultado generado por el programa se reporta en las carpetas `comprasConTDCMayorDe1500` y `comprasConTDCMenorIgualDe1500`

Explicación Código Script

```
11 def process_line(line):
12     |
13     | except ValueError:
14     |     gasto = 0.0
15     |     return (persona, metodo, gasto)
16     |
17     | else:
18     |     return None
19
20 def not_none(x):
21     | return x is not None
22
23 def filter_tarjeta(x):
24     | return "tarjeta" in x[1]
25
26 def gasto_mayor_1500(x):
27     | return x[2] > 1500
28
29 def gasto_menor_igual_1500(x):
30     | return x[2] <= 1500
31
32 def map_persona_uno(x):
33     | return (x[0], 1)
34
35 def sum_values(a, b):
36     | return a + b
37
38 def format_output(x):
39     | return f"{x[0]};{x[1]}"
40
41 processed_rdd = rdd.map(process_line).filter(not_none)
42
43 # Cacheamos porque lo usamos 2 veces
44 tdc_rdd = processed_rdd.filter(filter_tarjeta).cache()
45
46 # a) Compras con TDC > 1500
47 mayor_rdd = (
48     | tdc_rdd
49     | .filter(gasto_mayor_1500)
50     | .map(map_persona_uno)
51     | .reduceByKey(sum_values)
52 )
53
54 # b) Compras con TDC ≤ 1500
55 menor_rdd = (
56     | tdc_rdd
57     | .filter(gasto_menor_igual_1500)
58     | .map(map_persona_uno)
59     | .reduceByKey(sum_values)
60 )
61
62 # Convertir a texto
63 mayor_rdd_text = mayor_rdd.map(format_output)
64 menor_rdd_text = menor_rdd.map(format_output)
65
66 # Guardar resultados
67 mayor_rdd_text.saveAsTextFile(output_folder_mayor)
68 menor_rdd_text.saveAsTextFile(output_folder_menor)
69
70 # Liberar memoria
71 tdc_rdd.unpersist()
72
73 sc.stop()
```

```
from pyspark import SparkContext

sc = SparkContext(appName="personaYMetodosDePago")

input_file = "/user/dani/entrada.txt"
output_folder_mayor = "/user/dani/comprasConTDCMayorDe1500"
output_folder_menor = "/user/dani/comprasConTDCMenorIgualDe1500"

rdd = sc.textFile(input_file)
```

El programa comienza creando un *SparkContext* llamado *sc* con el nombre de aplicación “personaYMetodosDePago”, lo que indica a *Spark* que se trata de una tarea de procesamiento distribuido. A continuación, se define la ruta del archivo de entrada en *input_file*, así como las carpetas de salida *output_folder_mayor* y *output_folder_menor* donde se guardarán los resultados.

Se utiliza *rdd* para cargar el contenido del archivo de texto especificado, de forma que cada línea del archivo se convierte en un elemento del RDD. Para transformar esas líneas en datos estructurados, se aplica la función *process_line* mediante la operación *map*. La función *process_line* recibe una línea de texto, la divide en partes usando el carácter punto y coma como separador y extrae tres valores: persona, metodo y gasto. Además, convierte el gasto en un número flotante y devuelve una tupla con esos tres valores. Si la línea está mal formada, devuelve *None*.

Para eliminar los posibles valores nulos generados por líneas incorrectas, se aplica un filtro con la función *not_none*. De esta forma, el RDD resultante llamado **processed_rdd** solo contiene tuplas con datos válidos en la forma (persona, metodo, gasto).

Posteriormente, el código necesita trabajar únicamente con las compras realizadas con tarjeta. Para eso, se filtra el RDD **processed_rdd** utilizando la función *filter_tarjeta*, que se asegura de mantener solo las tuplas en las que el campo metodo contiene la palabra “tarjeta”. Este filtrado da lugar al RDD **tdc_rdd**. Se invoca el método *cache* sobre **tdc_rdd** para que *Spark* conserve estos datos en memoria, optimizando el rendimiento porque se va a utilizar en dos cálculos distintos.

Para resolver la parte (a) del ejercicio, el programa filtra el RDD **tdc_rdd** empleando la función *gasto_mayor_1500*, de manera que solo queden las compras cuyo gasto supera los 1500 euros. Luego, se transforma cada elemento de este RDD en un par (persona, 1) usando la función *map_persona_uno*. Con la operación *reduceByKey* y la función *sum_values*, se suman estos unos para contar cuántas compras realizó cada persona con tarjeta y un gasto mayor de 1500. El resultado se guarda en **mayor_rdd**.

Para la parte (b), el código realiza un proceso similar. Filtra **tdc_rdd** con la función *gasto_menor_igual_1500*, que selecciona únicamente las compras cuyo gasto es igual o inferior a 1500 euros. Al igual que antes, se transforma en pares (persona, 1) y se suman las compras por persona, generando **menor_rdd**.

Una vez obtenidos los conteos, ambos RDDs **mayor_rdd** y **menor_rdd** se convierten en texto con la función *format_output*, que crea una cadena de texto con el nombre de la persona y el número de compras, separados por un punto y coma. Estas salidas se almacenan respectivamente en **mayor_rdd_text** y **menor_rdd_text**.

Finalmente, se escriben los resultados en las carpetas correspondientes mediante *saveAsTextFile*, liberando después la memoria cacheada con *unpersist* en **tdc_rdd**. Por último, se detiene el contexto *Spark* con *sc.stop()* para finalizar correctamente la ejecución del programa.

En conjunto, el código utiliza transformaciones y acciones de *Spark* de forma eficiente, procesando los datos en paralelo y evitando operaciones innecesarias, lo que permite calcular de manera óptima las compras realizadas con tarjeta de crédito, diferenciadas según el importe gastado.

Ejemplo:

```
Alice;Tarjeta de crédito;1800
Bob;Tarjeta de crédito;1000
Alice;Tarjeta de crédito;900
Carol;Bizum;2500
Bob;Tarjeta de crédito;1700
```

- **processed_rdd = rdd.map(process_line).filter(not_none)**

```
[
  "Alice;Tarjeta de crédito;1800",
  "Bob;Tarjeta de crédito;1000",
  "Alice;Tarjeta de crédito;900",
  "Carol;Bizum;2500",
  "Bob;Tarjeta de crédito;1700"
]
```

- **tdc_rdd = processed_rdd.filter(filter_tarjeta).cache()**

```
[
  ("Alice", "tarjeta de crédito", 1800.0),
  ("Bob", "tarjeta de crédito", 1000.0),
  ("Alice", "tarjeta de crédito", 900.0),
  ("Bob", "tarjeta de crédito", 1700.0)
]
```

- `mayor_rdd = tdc_rdd.filter(gasto_mayor_igual_1500) .map(map_persona_uno).reduceByKey(sum_values)`

1. **Filter**

```
[  
    ("Alice", "tarjeta de crédito", 1800.0),  
    ("Bob", "tarjeta de crédito", 1700.0)  
]
```

2. `Map(map_persona_uno)`

```
[  
    ("Alice", 1),  
    ("Bob", 1)  
]
```

3. `ReduceByKey`

```
[  
    ("Alice", 1),  
    ("Bob", 1)  
]
```

- `menor_rdd = tdc_rdd.filter(gasto_menor_igual_1500) .map(map_persona_uno).reduceByKey(sum_values)`

4. **Filter**

```
[  
    ("Bob", "tarjeta de crédito", 1000.0),  
    ("Alice", "tarjeta de crédito", 900.0)  
]
```

5. `Map(map_persona_uno)`

6.

```
[  
    ("Alice", 1),  
    ("Bob", 1)  
]
```

7. `ReduceByKey`

```
[  
    ("Alice", 1),  
    ("Bob", 1)  
]
```

- **mayor_rdd_text = mayor_rdd.map(format_output)**

```
[  
  "Alice;1",  
  "Bob;1"  
]
```

- **menor_rdd_text = menor_rdd.map(format_output)**

```
[  
  "Alice;1",  
  "Bob;1"  
]
```

- **Resultado (Carpeta comprasConTDCMayorDe1500)**

Alice;1

Bob;1

- **Resultado (Carpeta comprasConTDCMenorIgualDe1500)**

Alice;1

Bob;1

Ejecución

Seguimos los siguientes pasos para ejecutar el programa:

1. Ubicarnos en la ruta deseada (cd /home/dani/)
2. Dar permiso de ejecución a los scripts (chmod u+x personaYMetodosDePago.py)
3. Activamos el yarn: start-yarn.sh y el DFS: start-dfs.sh
4. Ejecutar el programa:

```
dani@dani-VMware-Virtual-Platform:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
dani@dani-VMware-Virtual-Platform:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [dani-VMware-Virtual-Platform]
```

```
dani@dani-VMware-Virtual-Platform:~$ spark-submit --master yarn personaYMetodosDePago.py
Archivos 02:06:16 WARN Utils: Your hostname, dani-VMware-Virtual-Platform resolves to a loopback address: 127.0.1.1; using 192.168.176.131 instead (on interface ens33)
25/07/10 02:06:16 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
25/07/10 02:06:17 INFO SparkContext: Running Spark version 3.4.4
25/07/10 02:06:17 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
25/07/10 02:06:17 INFO ResourceUtils: =====
=====
25/07/10 02:06:17 INFO ResourceUtils: No custom resources configured for spark.driver.
25/07/10 02:06:17 INFO ResourceUtils: =====
=====
25/07/10 02:06:17 INFO SparkContext: Submitted application: personaYMetodosDePago
25/07/10 02:06:17 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(cores -> name: cores, amount: 1, script: , vendor: , memory -> name: memory, amount: 1024, script: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor: ), task resources: Map(cpus -> name: cpus, amount: 1.0)
25/07/10 02:06:17 INFO ResourceProfile: Limiting resource is cpus at 1 tasks per
```

Pasamos los datos obtenidos del Hadoop File System a local:

```
dani@dani-VMware-Virtual-Platform:~$ hdfs dfs -get /user/dani/comprasConTDCMayorDe1500 /home/dani/comprasConTDCMayorDe1500
dani@dani-VMware-Virtual-Platform:~$ hdfs dfs -get /user/dani/comprasConTDCMenorIgualDe1500 /home/dani/comprasConTDCMenorIgualDe1500
dani@dani-VMware-Virtual-Platform:~$
```

Resultado

- **Resultado (Carpeta comprasConTDCMayorDe1500)**

Alice;2

- **Resultado (Carpeta comprasConTDCMenorIgualDe1500)**

Alice;1

Bob;1